

PYTHON STANDARD LIBRARY

Six Essential Modules In Depth

A thorough, practical reference for `csv`, `json`, `random`, `time`, `datetime`, and `pathlib` — concepts, full function tables, worked examples, real-world patterns, and gotchas for each.

`csv`

`json`

`random`

`time`

`datetime`

`pathlib`

```
01 csv — reading & writing tabular files
02 json — serializing structured data
03 random — pseudo-random numbers & sampling
04 time — low-level clock & timing
05 datetime — calendar dates & arithmetic
06 pathlib — object-oriented filesystem paths
```

The **csv** module reads and writes CSV (comma-separated values) files — the plain-text spreadsheet format used everywhere for data exports. It correctly handles quoted fields, embedded commas, embedded newlines, and different delimiters, which is far safer than manually splitting lines on a comma.

CORE CONCEPTS

A CSV file is plain text: each line is a row, and a delimiter (usually a comma) separates columns. If a field contains that delimiter or a newline, it's wrapped in quotes (e.g. **"Smith, John"**). The module parses these rules via "dialects" — presets for delimiter, quote character, and line-ending behavior. The default dialect is **excel**.

KEY FUNCTIONS & CLASSES

<code>csv.reader(f, delimiter=',')</code>	Reads rows as lists of strings; delimiter can be changed (e.g. <code>'\t'</code> for TSV).
<code>csv.writer(f)</code>	Writes rows (lists/tuples) to a file as CSV text.
<code>csv.DictReader(f)</code>	Reads rows as ordered dictionaries, using the header row as keys.
<code>csv.DictWriter(f, fieldnames)</code>	Writes dictionaries as CSV rows in a defined column order.
<code>writer.writerow(row)</code>	Writes a single row.
<code>writer.writerows(rows)</code>	Writes multiple rows at once from a list of lists.
<code>writer.writeheader()</code>	(DictWriter) Writes the header row from fieldnames.
<code>csv.Sniffer()</code>	Auto-detects the dialect (delimiter, quoting) of an unknown CSV file.
<code>quoting=csv.QUOTE_ALL</code>	Forces quotes around every field when writing.
<code>restkey / restval</code>	DictReader options for handling rows with extra or missing columns.

READING & WRITING

<pre># reading with DictReader import csv with open("people.csv", newline="") as f: reader = csv.DictReader(f) for row in reader: print(row["name"], row["age"])</pre>	<pre># writing with DictWriter import csv rows = [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}] with open("out.csv", "w", newline="") as f: w = csv.DictWriter(f, fieldnames=["name", "age"]) w.writeheader() w.writerows(rows)</pre>
---	---

CUSTOM DELIMITERS (TSV EXAMPLE)

```
import csv

with open("data.tsv", newline="") as f:
    reader = csv.reader(f, delimiter="\t")
    for row in reader:
        print(row)
```

AUTO-DETECTING A DIALECT

```
import csv

with open("mystery.csv", newline="") as f:
    sample = f.read(2048)
    f.seek(0)
    dialect = csv.Sniffer().sniff(sample)
    reader = csv.reader(f, dialect)
    for row in reader:
        print(row)
```

FILTERING & TRANSFORMING ROWS

```
# keep only rows where age > 21, write to a new file
import csv

with open("people.csv", newline="") as fin, \
    open("adults.csv", "w", newline="") as fout:
    reader = csv.DictReader(fin)
    writer = csv.DictWriter(fout, fieldnames=reader.fieldnames)
    writer.writeheader()
    for row in reader:
        if int(row["age"]) > 21:
            writer.writerow(row)
```

Gotcha: Always open files with `newline=""` when reading/writing CSV — otherwise extra blank lines can appear between rows on Windows because both Python's text mode and the csv module try to translate line endings.

Tip: All values from `csv.reader/DictReader` come back as strings, even numbers — you must convert them yourself (e.g. `int(row["age"])`).

Use it for: importing/exporting spreadsheet data, logging structured records, ETL pipelines, or moving data between tools like Excel, Google Sheets, and Python.

The **json** module converts between Python objects (dicts, lists, strings, numbers, booleans) and JSON text — the standard format for web APIs, config files, and data interchange. JSON maps closely to Python's own data structures, making this module very lightweight to use.

CORE CONCEPTS

"Serializing" (dumping) turns a Python object into a JSON string; "deserializing" (loading) turns JSON text back into Python objects. Type mapping is mostly 1-to-1: dict ↔ object, list ↔ array, str ↔ string, int/float ↔ number, True/False ↔ true/false, None ↔ null. Not every Python type is JSON-serializable by default (e.g. datetime, sets, custom classes) — those need custom encoders.

KEY FUNCTIONS

<code>json.dumps(obj)</code>	Converts a Python object to a JSON-formatted string.
<code>json.loads(s)</code>	Parses a JSON string into a Python object.
<code>json.dump(obj, f)</code>	Writes a Python object as JSON directly to a file.
<code>json.load(f)</code>	Reads JSON from a file and parses it into a Python object.
<code>indent=N</code>	Pretty-prints output with N spaces of indentation.
<code>sort_keys=True</code>	Outputs dictionary keys in sorted order (useful for diffs/tests).
<code>separators=(',', ':')</code>	Customizes item/key separators; use for compact output.
<code>default=fn</code>	Custom function to serialize otherwise non-JSON-safe types.
<code>object_hook=fn</code>	Custom function to transform parsed dicts back into objects.
<code>json.JSONDecodeError</code>	Exception raised when parsing invalid JSON text.

BASIC SERIALIZATION

```
# Python -> JSON
import json

data = {"name": "Alice", "age": 30,
        "tags": ["a", "b"]}

text = json.dumps(data, indent=2)
print(text)
```

```
# JSON -> Python
import json

text = '{"name": "Alice", "age": 30}'
parsed = json.loads(text)
print(parsed["name"]) # Alice
```

FILES & CUSTOM TYPES

```
import json
from datetime import datetime

# write / read JSON files, handling non-serializable types
with open("data.json", "w") as f:
    json.dump({"created": datetime.now()}, f,
              default=lambda o: o.isoformat())

with open("data.json") as f:
    loaded = json.load(f)
```

CUSTOM ENCODING A CLASS

```
import json

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

def encode_point(obj):
    if isinstance(obj, Point):
        return {"x": obj.x, "y": obj.y}
    raise TypeError(f"Cannot serialize {type(obj)}")

json.dumps(Point(1, 2), default=encode_point)
```

HANDLING PARSE ERRORS

```
import json

try:
    data = json.loads('{ "bad": }')
except json.JSONDecodeError as e:
    print(f"Invalid JSON at line {e.lineno}: {e.msg}")
```

Gotcha: `json.dumps()` raises **`TypeError`** on objects it doesn't know how to serialize (like `datetime` or a custom class) — use **`default=`** or convert the value manually first. Also, JSON has no "tuple" type — tuples are serialized as arrays and come back as lists.

Use it for: talking to web APIs, saving config/settings files, caching structured data, and exchanging data between services in a human-readable format.

The **random** module generates pseudo-random numbers and makes random selections — useful for simulations, games, sampling, testing, and shuffling data. It's "pseudo-random" because the numbers come from a deterministic algorithm seeded by a starting value.

CORE CONCEPTS

Every run uses an internal generator (Mersenne Twister) seeded automatically from system entropy, so results differ each run by default. Call **random.seed(n)** to make results reproducible — useful for tests or demos. This module is **not cryptographically secure**; for security-sensitive randomness (tokens, passwords) use the **secrets** module instead.

KEY FUNCTIONS

<code>random.seed(n)</code>	Sets the starting seed so results become reproducible.
<code>random.random()</code>	Returns a random float between 0.0 and 1.0.
<code>random.uniform(a, b)</code>	Returns a random float between a and b.
<code>random.randint(a, b)</code>	Returns a random integer between a and b (inclusive).
<code>random.randrange(start, stop, step)</code>	Like <code>range()</code> , but returns one random value from it.
<code>random.choice(seq)</code>	Returns a single random element from a sequence.
<code>random.choices(seq, k=n, weights=w)</code>	Returns n random elements, with replacement; supports weighting.
<code>random.sample(seq, k)</code>	Returns k unique random elements, without replacement.
<code>random.shuffle(list)</code>	Shuffles a list in place, randomizing its order.
<code>random.gauss(mu, sigma)</code>	Returns a value from a normal (Gaussian) distribution.

EXAMPLES

```
# basic selection & shuffling
import random

dice_roll = random.randint(1, 6)
winner = random.choice(["Alice", "Bob"])

cards = ["A", "K", "Q", "J"]
random.shuffle(cards)
```

```
# sampling & weighted choice
import random

lotto = random.sample(range(1, 50), 6)

pick = random.choices(
    ["common", "rare"],
    weights=[90, 10], k=1
)
```

REPRODUCIBLE RESULTS (SEEDING)

```
import random

random.seed(42)
print(random.randint(1, 100)) # same output every run
print(random.randint(1, 100)) # stays consistent, in sequence
```

SIMULATING A WEIGHTED LOOT TABLE

```
import random

loot_table = {"common": 70, "rare": 25, "legendary": 5}

drop = random.choices(
    list(loot_table.keys()),
    weights=list(loot_table.values()),
    k=1
)[0]
print(drop)
```

MONTE CARLO STYLE SIMULATION

```
# estimate probability of rolling doubles on two dice
import random

trials = 100_000
doubles = 0
for _ in range(trials):
    a, b = random.randint(1, 6), random.randint(1, 6)
    if a == b:
        doubles += 1

print(doubles / trials) # approx 0.1667
```

Gotcha: Never use the **random** module for passwords, tokens, or anything security-related — it's predictable if the seed is known. Use **secrets.token_hex()** or **secrets.choice()** instead.

Tip: **random.sample()** raises an error if **k** is larger than the population and **replace=False** semantics apply — use **random.choices()** instead if you need repeats allowed.

Use it for: games, shuffling quiz questions, randomized test data, Monte Carlo simulations, and A/B test assignment.

The **time** module works with low-level time functions — measuring elapsed time, pausing execution, and reading the system clock. It's the older, closer-to-the-OS counterpart to the higher-level **datetime** module.

CORE CONCEPTS

Most of this module revolves around the Unix "epoch" — the number of seconds since January 1, 1970 UTC. **time.time()** gives you that number, which is easy to store or compare but not human-readable on its own. For accurate benchmarking, use **time.perf_counter()** rather than **time.time()**, since it isn't affected by system clock adjustments.

KEY FUNCTIONS

<code>time.time()</code>	Returns the current time in seconds since the Unix epoch.
<code>time.sleep(s)</code>	Pauses program execution for s seconds.
<code>time.perf_counter()</code>	High-precision timer, ideal for measuring code duration (not wall-clock).
<code>time.monotonic()</code>	A clock that never goes backwards; good for measuring intervals safely.
<code>time.process_time()</code>	Measures CPU time used by the process (excludes sleep time).
<code>time.localtime()</code>	Converts epoch seconds into a struct_time in local time.
<code>time.gmtime()</code>	Converts epoch seconds into a struct_time in UTC.
<code>time.strftime(fmt, t)</code>	Formats a struct_time (or current time) as a string.
<code>time.strptime(s, fmt)</code>	Parses a string into a struct_time object.
<code>time.ctime(ts)</code>	Converts epoch seconds into a human-readable string.

TIMING CODE & FORMATTING

```
# benchmarking a block
import time

start = time.perf_counter()
time.sleep(1)
elapsed = time.perf_counter() - start

print(f"Took {elapsed:.2f}s")
```

```
# formatting the clock
import time

now = time.localtime()
print(time.strftime(
    "%Y-%m-%d %H:%M:%S", now
))
```


EPOCH TIMESTAMPS

```
import time

ts = time.time()          # e.g. 1799472000.123456
readable = time.ctime(ts) # human readable string
print(readable)
```

SIMPLE RETRY LOOP WITH BACKOFF

```
import time

def call_with_retry(fn, max_attempts=5):
    delay = 1
    for attempt in range(max_attempts):
        try:
            return fn()
        except Exception:
            time.sleep(delay)
            delay *= 2 # exponential backoff
    raise RuntimeError("All retries failed")
```

COMPARING THE TIMING FUNCTIONS

<code>time.time()</code>	Wall-clock time; can jump if system clock changes. Good for timestamps.
<code>time.monotonic()</code>	Never goes backward; best for measuring elapsed intervals reliably.
<code>time.perf_counter()</code>	Highest resolution; best for precise benchmarking of short code blocks.

Gotcha: `time.time()` can jump backward or forward if the system clock is adjusted (NTP sync, manual change). For measuring durations, prefer `time.perf_counter()` or `time.monotonic()` instead.

Use it for: benchmarking code, adding delays/retries, rate-limiting, and low-level timestamping at the system level.

The **datetime** module provides classes for working with calendar dates and times — creating, formatting, comparing, and doing arithmetic on dates in a much more readable, structured way than the raw **time** module.

CORE CONCEPTS

Four main classes matter: **date** (year/month/day only), **time** (hour/minute/second only), **datetime** (both combined), and **timedelta** (a duration used for arithmetic between dates). Datetimes can be "naive" (no timezone info) or "aware" (carry a timezone) — mixing the two in comparisons raises an error, so pick one approach per project.

KEY FUNCTIONS & CLASSES

<code>datetime.now()</code>	Returns the current local date and time (naive).
<code>datetime.now(tz)</code>	Returns the current date and time in a given timezone (aware).
<code>datetime.date(y, m, d)</code>	Creates a date object for a specific calendar day.
<code>datetime.strftime(fmt)</code>	Formats a datetime object as a string.
<code>datetime.strptime(s, fmt)</code>	Parses a string into a datetime object.
<code>datetime.fromisoformat(s)</code>	Parses an ISO 8601 string (e.g. "2026-09-09") directly.
<code>timedelta(days=, hours=)</code>	Represents a duration, used for date/time arithmetic.
<code>dt.weekday()</code> / <code>isoweekday()</code>	Returns the day of the week as an integer.
<code>dt.replace(...)</code>	Returns a new datetime with specific fields changed.
<code>dt.timestamp()</code>	Converts a datetime object into Unix epoch seconds.

ARITHMETIC & FORMATTING

```
# date arithmetic
from datetime import datetime, timedelta

now = datetime.now()
next_week = now + timedelta(days=7)
diff = next_week - now
print(diff.days) # 7
```

```
# formatting & parsing
from datetime import datetime

now = datetime.now()
text = now.strftime("%B %d, %Y")

parsed = datetime.strptime(
    "2026-09-09", "%Y-%m-%d")
```



COMMON FORMAT CODES

<code>%Y / %m / %d</code>	4-digit year / 2-digit month / 2-digit day.
<code>%H / %M / %S</code>	24-hour, minute, second.
<code>%B / %b</code>	Full month name ("September") / abbreviated ("Sep").
<code>%A / %a</code>	Full weekday name ("Wednesday") / abbreviated ("Wed").
<code>%p</code>	AM/PM (used with %I for 12-hour format).

WORKING WITH TIMEZONES

```
from datetime import datetime, timezone, timedelta

# timezone-aware "now" in UTC
utc_now = datetime.now(timezone.utc)

# a fixed offset timezone, e.g. UTC+2 (Cairo)
cairo_tz = timezone(timedelta(hours=2))
local_now = datetime.now(cairo_tz)

print(utc_now.isoformat())
```

CALCULATING SOMEONE'S AGE

```
from datetime import date

def age(birthdate):
    today = date.today()
    years = today.year - birthdate.year
    if (today.month, today.day) < (birthdate.month, birthdate.day):
        years -= 1
    return years

print(age(date(2000, 5, 17)))
```

Gotcha: Comparing a naive datetime to an aware one raises **TypeError**. Be consistent — if you use timezones anywhere, use **datetime.now(timezone.utc)** everywhere in that project.

Use it for: scheduling, age/duration calculations, formatting human-readable dates, and parsing date strings from files, logs, or APIs.

The **pathlib** module offers an object-oriented way to handle filesystem paths — a modern, cross-platform replacement for string-based paths and the older **os.path** module. Paths become objects with useful methods instead of plain strings you manipulate manually.

CORE CONCEPTS

Path objects support the `"/` operator to join path segments, automatically using the correct separator for the current OS (backslash on Windows, forward slash elsewhere). This avoids bugs from hardcoding `"/` or `"\"` and makes path-building code far more readable than **os.path.join()** chains.

KEY FUNCTIONS & METHODS

<code>Path("x")</code>	Creates a path object representing a file or folder.
<code>Path.cwd()</code>	Returns the current working directory as a Path.
<code>Path.home()</code>	Returns the user's home directory as a Path.
<code>Path.exists()</code>	Checks whether the path exists on disk.
<code>Path.is_file()</code> / <code>is_dir()</code>	Checks whether the path is a file or a directory.
<code>Path.mkdir(parents=True)</code>	Creates a new directory, optionally creating parents too.
<code>Path.glob(pattern)</code>	Finds files matching a pattern, e.g. <code>"*.txt"</code> .
<code>Path.rglob(pattern)</code>	Like glob, but searches recursively through subfolders.
<code>Path.read_text()</code> / <code>write_text()</code>	Reads or writes a file's text content directly.
<code>Path.suffix</code> / <code>.stem</code> / <code>.name</code>	Extension, filename without extension, and full filename.
<code>Path.resolve()</code>	Returns the absolute, fully-resolved version of the path.

BUILDING & INSPECTING PATHS

```
# building paths
from pathlib import Path

folder = Path("documents")
folder.mkdir(exist_ok=True)

file = folder / "notes.txt"
file.write_text("Hello, pathlib!")
```

```
# inspecting a path
from pathlib import Path

p = Path("documents/notes.txt")
print(p.name)    # notes.txt
print(p.stem)    # notes
print(p.suffix)  # .txt
print(p.parent)  # documents
```

SEARCHING DIRECTORIES

```
from pathlib import Path

folder = Path("documents")

# list all .txt files in this folder
for txt_file in folder.glob("*.txt"):
    print(txt_file.name)

# search recursively through all subfolders
for py_file in folder.rglob("*.py"):
    print(py_file)
```

ORGANIZING FILES BY EXTENSION

```
from pathlib import Path
import shutil

source = Path("downloads")
for file in source.iterdir():
    if file.is_file():
        target_dir = source / file.suffix.lstrip(".")
        target_dir.mkdir(exist_ok=True)
        shutil.move(str(file), str(target_dir / file.name))
```

PATH PROPERTIES REFERENCE

<code>Path.parts</code>	Tuple of all path components, e.g. ('docs', 'notes.txt').
<code>Path.parents</code>	Sequence of ancestor directories, from nearest to root.
<code>Path.with_suffix('.md')</code>	Returns a new path with the extension changed.
<code>Path.with_name('x.txt')</code>	Returns a new path with the filename replaced.

Gotcha: Path objects aren't automatically strings. Some older libraries expect a plain string, so you may need to wrap it explicitly with `str(path)` when passing it to such functions.

Tip: Use `Path.iterdir()` for a flat, one-level listing of a directory's contents (files and folders), as an alternative to `glob("**")`.

Use it for: building, checking, and manipulating file paths in a readable, cross-platform way; walking directory trees; and reading/writing small files without manual `open()/close()` calls.